ITSAR2 313 – SYSTEM INTEGRATION AND ARCHITECTURE 2

LABORATORY REPORT: EXERCISE 3 — SYSTEMS ON BUSINESS LOGIC

Sagum, Patrick Ruiz.; Henson, Princess Terana Caram Rasonable;
Gargarita, Trisha Faith Casiano; Mogat, Ela Mae Trojillo;
Tibo-oc, Paul Felippe Gelle
Information Technology, College of Computer Studies
Carlos Hilado Memorial State University

Abstract

This laboratory exercise built a three-layer Product Ordering API using Node.js and Express. Business logic was isolated entirely in the controller layer to enforce five sequential rules: field presence validation, quantity validation, product existence, stock availability, and stock deduction on successful orders. Thirteen curl test commands verified all business rules against five products seeded in a JSON data file. The key finding is that separating business logic from the presentation and data layers prevents invalid data from reaching the data layer and makes rules independently modifiable without affecting the other layers.

I. INTRODUCTION

Business logic refers to the rules and processes that control how data is created, modified, and validated within a system. It acts as the decision-making layer between the presentation layer and the data layer. In this lab, the presentation layer is the curl client sending HTTP requests, the business logic layer is the Express controller enforcing order rules, and the data layer is the products.json file storing product information and stock levels.

Placing business logic in the UI layer makes validation dependent on the client, which can be bypassed. A curl command or Postman request can send any payload directly to the server regardless of what the UI enforces. Without server-side business logic, invalid data reaches the data layer unfiltered. The controller layer is the correct location because it intercepts every request regardless of how it was sent.

A common business rule in an online store is that a customer cannot place an order if stock is zero or if the requested quantity exceeds available inventory. This lab implements exactly those rules. When a customer requests 9999 units of a product with only 10 in stock, the business logic layer rejects the order before the data layer is touched. The data layer never reaches an invalid state because the controller acts as a gatekeeper.

II. METHODOLOGY / IMPLEMENTATION

The system separates concerns across three layers following the architecture described in the lab objectives. The presentation layer consists of curl commands that send HTTP requests. The business logic layer is implemented in controllers/orderController.js — the only file in the project that makes data decisions. The data layer is data/products.json, a JSON file that stores the five products and their current stock values. When an order succeeds, the controller writes the updated stock back to the file using fs.writeFileSync.

All five business rules execute sequentially inside the placeOrder() method. The order of execution is deliberate — field validation runs first so the controller never attempts to query the data layer with undefined values. Quantity validation runs second so invalid numbers are rejected before a product lookup is performed. Product existence is verified third. Stock availability is checked fourth and fifth. Each rule returns a specific ERROR_CODE and HTTP status code immediately, stopping further execution.

Rule 1 (VALIDATION_ERROR, 400) rejects requests missing product_id or quantity. Rule 2 (INVALID_QUANTITY, 400) rejects zero, negative, and non-integer quantities. Rule 3 (PRODUCT_NOT_FOUND, 404) rejects orders for products that do not exist in the data file. Rule 4 (OUT_OF_STOCK, 400) rejects orders when stock is already zero. Rule 5 (STOCK_EXCEEDED, 400) rejects orders when the requested quantity exceeds available stock. If all rules pass, stock is deducted, the file is updated, and a 201 response is returned with the order summary.

```
curl -i -X POST http://localhost:3000/api/orders
  -H "Content-Type: application/json"
  -d '{"product_id":1,"quantity":2}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1 (main)
$  cd "c:\Users\parus\Desktop\sar-lab1\lab3\server" && npm start

> lab3-business-logic-api@1.0.0 start
> node server.js

Lab 3 API running on http://localhost:3000
```

Step 1 — GET all products (200)

curl -i http://localhost:3000/api/products

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$      curl -i http://localhost:3000/api/products
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 284
ETag: W/"11c-4qY5LihjA5EMzJrdZtwm5LgLJAg"
Date: Sun, 15 Mar 2026 05:50:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"data":[{"id":1,"name":"Laptop","price":45999,"stock":10},{"id":2,"name":"Smartphone","price":18999,"stock":25},{"id":3,"name":"Tablet","price":1
2499,"stock":15},{"id":4,"name":"Monitor","price":9999,"stock":8},{"id":5,"name":"Keyboard","price":1299,"stock":50}],"message":"success"}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$
```

Step 2 — GET single product (200)

curl -i http://localhost:3000/api/products/1

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i http://localhost:3000/api/products/1
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 78
ETag: W/"4e-jqiRRe/HOr0qWS2irSDPmLWgZbQ"
Date: Sun, 15 Mar 2026 05:52:06 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"data":{"id":1,"name":"Laptop","price":45999,"stock":10},"message":"success"}
```

Step 3 — POST valid order (201)

```
curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":2}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":2}'
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 102
ETag: W/"66-aiC1HcGwU8fFYwJCPI9HtRlEYKU"
Date: Sun, 15 Mar 2026 05:53:03 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Order successful","product":"Laptop","quantity":2,"total_price":91998,"remaining_stock":8}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$
```

Step 4 — GET product after order — stock reduced (200)

curl -i http://localhost:3000/api/products/1

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i http://localhost:3000/api/products/1
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-XSw136Zq5bCYRTOsre83H3zOC4I"
Date: Sun, 15 Mar 2026 05:53:25 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"data":{"id":1,"name":"Laptop","price":45999,"stock":8},"message":"success"}
```

EDGE CASES

Step 5 — Invalid product ID (404)

curl -i -X POST http://localhost:3000/api/orders \

-H "Content-Type: application/json" \

-d '{"product_id":99,"quantity":1}'

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":99,"quantity":1}'
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 76
ETag: W/"4c-41vcEblPGsXCDi21gFTQl8sCQ+4"
Date: Sun, 15 Mar 2026 05:53:47 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"PRODUCT_NOT_FOUND","message":"Product with id 99 does not exist."}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
```

Step 6 — Quantity = 0 (400)

```
curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":0}'
```

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":0}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 76
ETag: W/"4c-TuVdrgWJCyHF1RR6SSMR8A+dUQI"
Date: Sun, 15 Mar 2026 05:54:13 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"INVALID_QUANTITY","message":"Quantity must be greater than zero."}
```

Step 7 — Negative quantity (400)

```
curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":-5}'
```

```
TERMINAL    PROBLEMS    OUTPUT    DEBUG CONSOLE    POSTMAN CONSOLE    EXPLORER    PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":-5}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 69
ETag: W/"45-sNvhWv9Q4ELszOYOtoEZGjUZndY"
Date: Sun, 15 Mar 2026 05:54:34 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"INVALID_QUANTITY","message":"Quantity cannot be negative."}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
```

Step 8 — Stock exceeded (400)

curl -i -X POST http://localhost:3000/api/orders \

-H "Content-Type: application/json" \

-d '{"product_id":1,"quantity":9999}'

```
TERMINAL    PROBLEMS    OUTPUT    DEBUG CONSOLE    POSTMAN CONSOLE    EXPLORER    PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
>   -H "Content-Type: application/json" \
>   -d '{"product_id":1,"quantity":9999}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 93
ETag: W/"5d-3V/9zfW6EtyotJmYIcmb7bqMn4k"
Date: Sun, 15 Mar 2026 05:54:57 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"STOCK_EXCEEDED","message":"Requested quantity (9999) exceeds available stock (8)."}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$
```

Step 9 — Missing product_id (400)

curl -i -X POST http://localhost:3000/api/orders \

-H "Content-Type: application/json" \

-d '{"quantity":1}'

```
TERMINAL    PROBLEMS    OUTPUT    DEBUG CONSOLE    POSTMAN CONSOLE    EXPLORER    PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
>   -H "Content-Type: application/json" \
>   -d '{"quantity":1}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 74
ETag: W/"4a-vB5JnTbBcBMIBUH82Bx82HgGJZ0"
Date: Sun, 15 Mar 2026 05:55:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"VALIDATION_ERROR","message":"The product_id field is required."}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
```

Step 10 — Missing quantity (400)

curl -i -X POST http://localhost:3000/api/orders \

-H "Content-Type: application/json" \

-d '{"product_id":1}'

```
TERMINAL    PROBLEMS    OUTPUT    DEBUG CONSOLE    POSTMAN CONSOLE    EXPLORER    PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 72
ETag: W/"48-bHSp4sJxMvhh0iHX7bRyGf41nkM"
Date: Sun, 15 Mar 2026 05:55:35 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"VALIDATION_ERROR","message":"The quantity field is required."}
```

Step 11 — Empty body (400)

curl -i -X POST http://localhost:3000/api/orders \

-H "Content-Type: application/json" \

-d '{}'

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
>   -H "Content-Type: application/json" \
>   -d '{}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 74
ETag: W/"4a-vB5JnTbBcBMIBUH82Bx82HgGJZ0"
Date: Sun, 15 Mar 2026 05:55:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"VALIDATION_ERROR","message":"The product_id field is required."}
```

Step 12 — Out of stock (400)

Drain Monitor (product 4, stock=8) to zero first:

curl -i -X POST http://localhost:3000/api/orders \

  -H "Content-Type: application/json" \

  -d '{"product_id":4,"quantity":8}'

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":4,"quantity":8}'
HTTP/1.1 201 Created
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 103
ETag: W/"67-HL40n+JYdd/dh15gGAKK/9kfrnc"
Date: Sun, 15 Mar 2026 05:56:40 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Order successful","product":"Monitor","quantity":8,"total_price":79992,"remaining_stock":0}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
```

Then attempt to order from empty stock:

curl -i -X POST http://localhost:3000/api/orders \

  -H "Content-Type: application/json" \

  -d '{"product_id":4,"quantity":1}'

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i -X POST http://localhost:3000/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":4,"quantity":1}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 61
ETag: W/"3d-ZziG5KLf8L/rs6M8gm4BjU68Vkk"
Date: Sun, 15 Mar 2026 05:57:01 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"OUT_OF_STOCK","message":"Product is out of stock."}
Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
```

Step 13 — Unknown route (404)

curl -i http://localhost:3000/api/nonexistent

```
TERMINAL   PROBLEMS   OUTPUT   DEBUG CONSOLE   POSTMAN CONSOLE   EXPLORER   PORTS

Patrick R. Sagum@TFROG MINGW64 ~/Desktop/sar-lab1/lab3/server (main)
$ curl -i http://localhost:3000/api/nonexistent
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 54
ETag: W/"36-6OewuKwjAK+2g9plkoXF2roJMI4"
Date: Sun, 15 Mar 2026 05:57:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"NOT_FOUND","message":"Route does not exist"}
```

III. EXPERIMENTAL FINDINGS / OBSERVATIONS

All 13 test steps returned the correct HTTP status codes. Steps 1 through 4 confirmed the happy path: products are listed correctly, individual products are retrievable by ID, valid orders are processed and stock is deducted, and the updated stock is immediately visible in subsequent GET requests. Step 4 explicitly confirmed that the data layer was updated — Laptop stock dropped from 10 to 8 after the order in Step 3.

Steps 5 through 13 confirmed all edge cases. Invalid product IDs return 404 PRODUCT_NOT_FOUND. Zero, negative, and out-of-range quantities return 400 INVALID_QUANTITY or 400 STOCK_EXCEEDED. Missing fields return 400 VALIDATION_ERROR. Step 12 confirmed the OUT_OF_STOCK rule by first draining Monitor stock to zero with a valid order, then attempting a second order against the empty stock. Step 13 confirmed that unknown routes return 404 NOT_FOUND with a JSON body rather than an HTML error page.

Step	Test Scenario	Expected	Actual	Result
1	GET all products	200	200	**PASS**
2	GET product by ID	200	200	**PASS**
3	POST valid order	201	201	**PASS**
4	GET product — stock reduced	200	200	**PASS**
5	Invalid product ID	404	404	**PASS**
6	Quantity = 0	400	400	**PASS**
7	Negative quantity	400	400	**PASS**
8	Stock exceeded	400	400	**PASS**
9	Missing product_id	400	400	**PASS**
10	Missing quantity	400	400	**PASS**
11	Empty body	400	400	**PASS**
12	Out of stock	400	400	**PASS**
13	Unknown route	404	404	**PASS**

IV. LAB DISCUSSIONS / COMPUTATIONS

Without the five business rules, a client could submit quantity 9999 for a product with only 10 in stock. The data layer would deduct 9999 from 10 and write -9989 back to the file — a permanently invalid state. Business logic acts as a gatekeeper that prevents the data layer from ever storing invalid values. Every write to products.json is preceded by validation that guarantees the resulting stock value will be zero or positive.

The UI can be bypassed entirely. A curl command or Postman request sends payloads directly to the server without passing through any frontend validation. If the controller contains no rules, every request reaches the data layer regardless of content. Negative quantities, nonexistent product IDs, and orders that exceed stock all modify the data file without error. The system becomes inconsistent and the data cannot be trusted.

The route file only maps URLs to handlers — it has no access to business context. The data file stores values but has no knowledge of what constitutes a valid order. The controller sits between the two and has access to both the incoming request and the data layer, making it the only appropriate location for decision-making. This mirrors the Repository pattern from Lab 1 and Lab 2 where the controller delegates to a single layer that owns all data interactions.

The orderController.js in this lab plays the same architectural role as the Repository layer in Lab 1 and Lab 2. Both are the single location where all data decisions are made. In Lab 2, the Repository Interface was the gatekeeper between the controller and Eloquent. In this lab, the controller itself is the gatekeeper between the HTTP request and the JSON file. The principle is the same: centralize data decisions in one place so they can be audited, tested, and modified without touching other layers.

V. CONCLUSIONS

Business logic was successfully isolated in the controller layer and all five rules were implemented and verified across 13 test cases. The three-layer separation — curl as presentation, Express controller as business logic, products.json as data — demonstrated that each layer can be modified independently without affecting the others. The data layer never received an invalid write because the controller intercepted and rejected every invalid request before reaching fs.writeFileSync. In a production system the data layer would be replaced by a relational database and the products.json helper functions would be replaced by a Repository class, but the business logic in the controller would remain unchanged. This confirms that proper architectural separation reduces the cost of future changes and prevents data integrity failures at the source.

References

OpenJS Foundation. (2024). Node.js documentation. https://nodejs.org/en/docs

Express.js Contributors. (2024). Express: Fast, unopinionated, minimalist web framework for Node.js. https://expressjs.com

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Mozilla Developer Network. (2024). HTTP response status codes. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Stenberg, D. (2024). curl: Command line tool and library for transferring